

Introduction à Python intermédiaire

Python intermédiaire orienté traitements métier

- Public : profils métier ayant déjà suivi le cours débutant
- Objectif : automatiser, contrôler et fiabiliser des traitements de données
- Angle : notebook pour explorer, script .py pour produire

Cadrage

- Formation de 3 jours
- Niveau : intermédiaire
- Périmètre : Polars, notebook, Excel, requests, logging, debugger
- Hors périmètre : sujets avancés non nécessaires à ce parcours

Public et prérequis

- Avoir déjà manipulé variables, conditions, boucles et fonctions simples
- Savoir exécuter un script Python dans VS Code ou un terminal
- Comprendre le principe d'un fichier CSV
- Ne pas être développeur n'est pas un problème

Modalité pédagogique

- 30% théorie / 70% pratique
- Beaucoup de démonstration sur cas métier
- Un fil rouge unique sur les 3 jours
- Quiz flash et corrections collectives

Objectifs de sortie

À la fin du cours, les participants savent :

- lire un CSV et un Excel avec Polars
- créer des colonnes dérivées sans détruire la donnée source
- utiliser un notebook pour comprendre une transformation
- transformer cette logique en script .py
- appeler une API simple et joindre les données

Stack du cours

- Python 3.12+
- VS Code
- extension Python
- extension Jupyter
- polars
- altair comme backend de `df.plot`
- requests

Règles pédagogiques

- `import polars as pl`
- notebook = exploration et visualisation
- script .py = livrable principal
- colonnes sources conservées
- transformations via colonnes dérivées

Organisation des livrables

- `script_exploration.py`
- `script_final.py`
- `output/resultat_final.csv` ou `output/resultat_final.xlsx`
- `output/anomalies.csv`
- `README_execution.md`

Fil rouge métier

Cas fil rouge sur les 3 jours :

- un export brut à nettoyer
- un notebook pour explorer et contrôler
- un script pour rejouer le traitement
- une API pour enrichir les données
- un export final propre + anomalies

Plan des 3 jours

- Jour 1 : notebook, Polars, premières transformations
- Jour 2 : contrôle qualité, Excel, passage au script, debugger
- Jour 3 : gros volume, agrégations, jointure FX, API réelle, logging minimal, bonus debugger

Jour 1

**Notebook + Polars + premières
transformations**

Jour 1 - objectifs

- faire la transition depuis le niveau débutant
- comprendre le rôle du notebook
- découvrir Polars sur un cas utile
- construire un premier pipeline simple

Ce qu'on reprend du niveau débutant

- lecture de code simple
- variables et fonctions
- structures conditionnelles
- manipulation basique de chaînes

Comment lire un petit script de transformation

```
rows = lire_lignes("input/clients.csv")
rows_ok = [row for row in rows if row["statut"] == "ok"]

for row in rows_ok:
    row["email_clean"] = row["email"].lower()

ecrire_lignes("output/clients_ok.csv", rows_ok)
```

- entrée : input/clients.csv
- filtre : on garde seulement les lignes avec statut == "ok"
- transformation : on crée email_clean
- sortie : output/clients_ok.csv

Dans quel ordre lire ce script

1. repérer le fichier d'entrée
 2. repérer la condition de filtrage
 3. repérer la colonne créée ou modifiée
 4. repérer le fichier de sortie
- inutile de comprendre chaque détail au premier passage
 - au début, il faut surtout comprendre le trajet de la donnée

Boucle simple vs map / filter

```
emails = [" A@x.com ", " ", "B@Y.COM"]
```

```
emails_clean = []  
for email in emails:  
    if email.strip():  
        emails_clean.append(email.strip().lower())
```

- une boucle for est souvent la forme la plus lisible
- map transforme
- filter sélectionne
- il faut savoir les lire, pas en abuser

map

```
nombres = [1, 2, 3]
```

```
resultat = list(map(lambda x: x * 2, nombres))
```

- résultat : [2, 4, 6]
- utile pour transformer une liste simple

filter

```
nombres = [1, 2, 3, 4]
```

```
resultat = list(filter(lambda x: x % 2 == 0, nombres))
```

- résultat : [2, 4]
- utile pour garder seulement certains éléments

map / filter : quand c'est utile, quand éviter

- utile sur listes courtes et cas simples
- utile pour lire du code existant
- à éviter si la lisibilité baisse
- pour les tableaux métier, Polars sera souvent plus clair

Transition : de map / filter vers notebook puis Polars

- sur une liste, on transforme élément par élément
- le notebook sert à tester cette logique pas à pas
- sur un tableau métier, on raisonne ensuite en colonnes et en lignes
- Polars arrive quand la structure et le volume augmentent

Exercice J1-A - Réécrire une boucle avec filter puis map

Consigne :

- partir de cette liste :

```
emails = [" A@x.com ", " ", "B@Y.COM", " "]
```

- supprimer les valeurs vides après strip()
- mettre les emails restants en minuscules
- produire emails_clean

Correction J1-A

```
emails = [" A@x.com ", " ", "B@Y.COM", "  "]

emails_non_vides = list(filter(lambda email: email.strip(), emails))
emails_clean = list(map(lambda email: email.strip().lower(), emails_non_vides))
```

emails_clean

```
["a@x.com", "b@y.com"]
```

- `filter` enlève les valeurs vides
- `map` transforme les valeurs restantes
- le résultat est correct
- on verra juste après pourquoi ce style devient moins pratique sur un vrai tableau métier

Notebook vs script

```
# notebook
```

```
montants = [1200.5, 980.0, 450.0]
```

```
sum(montants)
```

```
# script
```

```
def main() -> None:
```

```
    montants = [1200.5, 980.0, 450.0]
```

```
    print(sum(montants))
```

- notebook : observer, tester, visualiser
- script : exécuter de bout en bout
- notebook : plus souple
- script : plus stable et plus réutilisable

Notebook dans VS Code

```
x = 10  
y = 20  
x + y
```

- créer un fichier .ipynb
- choisir l'interpréteur Python
- exécuter cellule par cellule
- relancer une cellule en gardant le contexte

Structure d'un notebook

```
# Cellule 1
```

```
fichier = "input/clients.csv"
```

```
# Cellule 2
```

```
colonnes_attendues = ["client", "email", "montant", "statut"]
```

```
# Cellule 3
```

```
len(colonnes_attendues)
```

- cellule markdown : expliquer
- cellule code : transformer
- cellule résultat : observer
- on travaille par étapes courtes

Ce qu'on met dans un notebook

- lecture de données
- aperçu du schéma
- vérifications rapides
- graphiques simples
- comparaison avant / après

Ce qu'on ne met pas dans un notebook

- toute la logique de production finale
- les secrets
- des copier-coller inutiles
- des cellules dans le désordre

Pourquoi Polars dans ce cours

- on manipule des tableaux de données
- on veut lire, filtrer, transformer et exporter rapidement
- Polars est la librairie tabulaire du cours intermédiaire
- on l'utilise de façon simple et lisible

DataFrame : notion clé

Un DataFrame, c'est :

- un tableau structuré
- avec des colonnes nommées
- des types de données
- des opérations de filtrage et de transformation

Exemple de colonnes :

- client
- email
- montant
- statut

Construire un petit DataFrame

```
import polars as pl

df_demo = pl.DataFrame(
    {
        "client": ["Alpha", "Beta"],
        "montant": [1200.50, 980.00],
        "statut": ["ok", "pending"],
    }
)
```

df_demo

- utile pour comprendre l'objet manipulé
- même logique qu'un tableau avec colonnes nommées

import polars as pl

```
import polars as pl
```

- convention unique du cours
- pl sera le préfixe utilisé partout dans les exemples

Lire un CSV avec Polars

```
import polars as pl
```

```
df = pl.read_csv("input/clients.csv")
```

- df est un DataFrame
- c'est notre point de départ pour l'analyse

Aperçu du DataFrame

`df.head()`

- voir les premières lignes
- comprendre la structure générale
- vérifier si les colonnes attendues sont là

Ce que montre head()

shape: (5, 4)

client	email	montant	statut
---	---	---	---
str	str	str	str

- shape = nombre de lignes et colonnes affichées
- les noms de colonnes sont visibles
- les types sont visibles

Schéma, types, nulls

```
df.schema
```

```
df.null_count()
```

- voir les types
- repérer les colonnes problématiques
- compter rapidement les valeurs manquantes

Résumer rapidement les colonnes avec `df.describe()`

`df.describe()`

- utile pour un premier diagnostic rapide
- particulièrement utile sur les colonnes numériques
- permet de voir min, max, moyenne et nombre de valeurs

Expression Polars : `pl.col(...)`

```
pl.col("email")
```

```
pl.col("montant")
```

```
pl.col("email").str.to_lowercase()
```

- une expression désigne une colonne ou une transformation
- seule, elle ne produit pas encore de résultat visible
- elle s'exécute dans un contexte comme `select`, `with_columns` ou `filter`

Les 3 contextes Polars à connaître

- `select` : choisir ou calculer des colonnes
- `with_columns` : ajouter des colonnes au DataFrame
- `filter` : garder seulement certaines lignes

select

```
df.select(  
    pl.col("client"),  
    pl.col("email"),  
    pl.col("statut"),  
)
```

- sélectionner seulement les colonnes utiles
- alléger l'analyse

filter avec Polars

```
df.filter(pl.col("statut") == "ok")
```

- garder uniquement les lignes utiles
- base du contrôle qualité

sort

```
df.sort("montant", descending=True)
```

- prioriser l'analyse
- faire émerger les cas extrêmes

Exercice J1-B - Explorer un export brut

Consigne :

- charger un CSV
- afficher les 5 premières lignes
- afficher le schéma
- repérer les colonnes à surveiller

Correction J1-B

```
import polars as pl

df = pl.read_csv("input/clients.csv")

print(df.head())
print(df.schema)
print(df.null_count())
print(df.describe())
```

- lecture du fichier
- aperçu rapide
- schéma vérifié
- colonnes avec valeurs manquantes repérées
- résumé statistique affiché

Créer des colonnes avec `with_columns`

```
df = df.with_columns(  
    pl.col("email").str.to_lowercase().alias("email_clean")  
)
```

- le cœur de la transformation métier
- `with_columns` ajoute des colonnes
- contrairement à `select`, on garde aussi les colonnes déjà présentes

Nettoyage texte

```
df = df.with_columns(  
    pl.col("email")  
        .str.strip_chars()  
        .str.to_lowercase()  
        .alias("email_clean")  
)
```

- strip_chars()
- to_lowercase()
- replace()
- normaliser avant de contrôler

Le pont avec `map` Python :

`map_elements()`

```
def clean_email(value: str) -> str:  
    return value.strip().lower()
```

```
df = df.with_columns(  
    pl.col("email")  
        .map_elements(clean_email, return_dtype=pl.String)  
        .alias("email_clean")  
)
```

- `map_elements()` applique une fonction Python valeur par valeur
- c'est le pont naturel avec `map(...)` vu plus tôt
- toujours préciser `return_dtype`

Préférer les expressions natives quand c'est possible

```
df = df.with_columns(  
    pl.col("email")  
        .str.strip_chars()  
        .str.to_lowercase()  
        .alias("email_clean")  
)
```

- ici, l'expression native est meilleure que `map_elements()`
- plus lisible
- plus rapide
- `map_elements()` sert surtout quand la logique n'existe pas déjà dans Polars

Normaliser un montant

```
df = df.with_columns(  
    pl.col("montant")  
        .str.replace_all("€", "")  
        .str.replace_all(" ", "")  
        .str.replace(",", ".")  
        .cast(pl.Float64, strict=False)  
        .alias("montant_net")  
)
```

- supprimer les espaces
- gérer , et .
- convertir en nombre
- ranger le résultat dans une nouvelle colonne

Calculer TVA et TTC avec des expressions Polars

```
TAUX_TVA = 0.20
```

```
df = df.with_columns([
    (pl.col("montant_net") * TAUX_TVA).round(2).alias("montant_tva"),
    (pl.col("montant_net") * (1 + TAUX_TVA)).round(2).alias("montant_ttc"),
])
```

- dans cet exemple, montant_net est notre base HT
- `pl.col("montant_net") * TAUX_TVA` calcule la TVA
- `pl.col("montant_net") * (1 + TAUX_TVA)` calcule le TTC
- on ajoute des colonnes métier sans écraser la source

Si la source est TTC, retrouver le HT

```
TAUX_TVA = 0.20
```

```
df = df.with_columns([
    (pl.col("montant_ttc") / (1 + TAUX_TVA)).round(2).alias("montant_ht"),
    (
        pl.col("montant_ttc")
        - (pl.col("montant_ttc") / (1 + TAUX_TVA))
    ).round(2).alias("montant_tva"),
])
```

- ici la colonne de départ est `montant_ttc`
- on reconstruit le HT
- on en déduit la TVA
- même logique : on ajoute des colonnes calculées

Exercice express - Calculer TVA et TTC

Consigne :

- partir d'une colonne montant_net considérée comme HT
- ajouter montant_tva
- ajouter montant_ttc
- utiliser un taux de 20%

Correction - Calculer TVA et TTC

```
TAUX_TVA = 0.20
```

```
df = df.with_columns([  
    (pl.col("montant_net") * TAUX_TVA).round(2).alias("montant_tva"),  
    (pl.col("montant_net") * (1 + TAUX_TVA)).round(2).alias("montant_ttc"),  
])
```

- une expression par colonne calculée
- résultat arrondi à 2 décimales
- aucun écrasement de la colonne source

Normaliser une date

```
df = df.with_columns(  
    pl.col("date_commande")  
        .str.strptime(pl.Date, format="%d/%m/%Y", strict=False)  
        .dt.strftime("%Y-%m-%d")  
        .alias("date_commande_clean")  
)
```

- identifier le format source
- convertir dans un format stable
- garder la colonne source si utile
- créer une colonne date propre

Normaliser un statut

```
df = df.with_columns(  
    pl.col("statut")  
        .str.strip_chars()  
        .str.to_lowercase()  
        .replace({"pending": "en_attente", "ok": "valide"})  
        .alias("statut_normalise")  
)
```

- mettre en minuscules
- supprimer les espaces parasites
- ramener plusieurs variantes vers une même valeur

Règle non négociable : colonnes dérivées

- on garde les colonnes sources
- on ajoute `email_clean`, `montant_net`, etc.
- on évite l'écrasement destructif
- on rend la transformation traçable

Assembler plusieurs colonnes dérivées dans un même pipeline

```
df = df.with_columns([
    pl.col("email").str.strip_chars().str.to_lowercase().alias("email_clean"),
    pl.col("montant").str.replace(",", ".").cast(pl.Float64, strict=False).alias("montant_net"),
    pl.col("statut").str.strip_chars().str.to_lowercase().alias("statut_normalise"),
])
```

- email_clean : email nettoyé
- montant_net : montant converti
- statut_normalise : statut harmonisé
- un seul with_columns peut porter plusieurs transformations lisibles

Exercice J1-C - Ajouter une colonne de contrôle au pipeline

Consigne :

- repartir du pipeline précédent
- ajouter `ligne_valide`
- `ligne_valide` vaut `True` si `email_clean` contient `@`
- `ligne_valide` vaut `True` si `montant_net` n'est pas nul
- garder les colonnes sources

Correction J1-C

```
df = df.with_columns([
    pl.col("email").str.strip_chars().str.to_lowercase().alias("email_clean"),
    pl.col("montant").str.replace(",", ".").cast(pl.Float64, strict=False).alias("montant_net"),
    pl.col("statut").str.strip_chars().str.to_lowercase().alias("statut_normalise"),
]).with_columns(
    (
        pl.col("email_clean").str.contains("@", literal=True)
        & pl.col("montant_net").is_not_null()
    ).alias("ligne_valide")
)
```

- transformation via with_columns
- conservation de la source
- logique de validation séparée
- sortie lisible

Atelier Jour 1 - Nettoyer un export métier simple

Objectif :

- lire un CSV
- créer les colonnes dérivées attendues
- produire une première table exploitable

Squelette atelier J1

- lecture du fichier
- analyse rapide
- ajout des colonnes
- vérification sur quelques lignes
- export d'une sortie intermédiaire

Livrable Jour 1

- notebook d'exploration propre
- première transformation exécutable
- sortie intermédiaire compréhensible

Quiz flash Jour 1

- rôle du notebook
- rôle du script
- utilité de `with_columns`
- calculer une colonne TVA ou TTC avec une expression
- retrouver un montant HT à partir d'un TTC
- pourquoi garder les colonnes sources

Récapitulatif Jour 1

- lire
- comprendre
- filtrer
- créer des colonnes dérivées
- faire un calcul métier simple avec des expressions Polars
- manipuler un cas HT / TVA / TTC
- préparer un vrai pipeline

Jour 2

**Contrôle qualité + Excel +
passage au script**

Jour 2 - objectifs

- mesurer la qualité des données
- visualiser les résultats
- intégrer Excel
- passer du notebook au script
- apprendre à déboguer

Rappel du fil rouge

- export brut
- nettoyage
- contrôle qualité
- export final
- future intégration API

Pourquoi faire du contrôle qualité

- vérifier qu'on ne produit pas de faux résultats
- repérer les anomalies métier
- expliquer la transformation
- sécuriser le livrable final

Compter rapidement les valeurs d'une colonne avec `value_counts()`

```
df["statut_normalise"].value_counts(sort=True)
```

- utile pour voir immédiatement la répartition d'une colonne catégorielle
- très pratique dans un notebook en phase d'exploration
- bon réflexe avant un graphique ou une règle métier

Quand utiliser `value_counts()` plutôt que `group_by`

- `value_counts()` : compter vite une seule colonne
- `group_by(...)` : calculer plusieurs indicateurs
- les deux répondent à des besoins proches
- `value_counts()` est souvent le point d'entrée le plus simple

Agréger = résumer un tableau

```
resume = df.select([
    pl.len().alias("nb_lignes"),
    pl.col("montant_net").sum().alias("montant_total"),
    pl.col("montant_net").mean().alias("montant_moyen"),
])
```

- on passe de milliers de lignes à quelques indicateurs
- on commence souvent par ce niveau de synthèse

Regrouper pour comparer des statuts

```
df.groupby("statut_normalise").agg([  
    pl.len().alias("nb_lignes"),  
    pl.col("montant_net").sum().alias("montant_total"),  
])
```

- base des synthèses métier

Exemple de sortie agrégée

statut_normalise	nb_lignes	montant_total
valide	120	185430.50
en_attente	34	24100.00

- une ligne par groupe
- pratique pour les contrôles et les graphiques

Quels indicateurs regarder d'abord

```
df.select([
    pl.len().alias("nb_lignes"),
    pl.col("montant_net").sum().alias("montant_total"),
    pl.col("montant_net").mean().alias("montant_moyen"),
])
```

- len : compter les lignes
- sum : sommer un indicateur
- mean : moyenne simple
- indispensables pour un mini diagnostic qualité

Vérifier qu'un client n'apparaît pas plusieurs fois

```
df.groupby("email_clean").agg(  
    pl.len().alias("nb_occurrences")  
).filter(pl.col("nb_occurrences") > 1)
```

- identifier une clé logique
- compter les répétitions
- distinguer doublon technique et métier

Mesurer ce qui bloque l'exploitation

```
df.select([
    pl.col("ligne_valide").not_().sum().alias("nb_invalides"),
    pl.col("email_clean").is_null().sum().alias("nb_emails_vides"),
    pl.col("montant_net").is_null().sum().alias("nb_montants_invalides"),
])
```

- lignes invalides
- dates absentes
- emails incorrects
- montants non exploitables

Créer une colonne métier conditionnelle avec `when / then / otherwise`

```
df = df.with_columns(  
    pl.when(pl.col("ligne_valide"))  
    .then(pl.lit("ok"))  
    .otherwise(pl.lit("a_corriger"))  
    .alias("statut_qualite")  
)
```

- équivalent d'un `if / else` sur une colonne
- très utile pour rendre un contrôle qualité lisible
- utiliser `pl.lit(...)` pour écrire une vraie valeur texte
- chaque branche doit être valide même si la condition ne passe pas

Gérer plusieurs cas avec when / then / otherwise

```
df = df.with_columns(  
    pl.when(pl.col("montant_net").is_null())  
      .then(pl.lit("montant_invalide"))  
      .when(  
          pl.col("email_clean").is_null()  
          | pl.col("email_clean").str.contains("@", literal=True).not_()  
      )  
      .then(pl.lit("email_invalide"))  
      .otherwise(pl.lit("ok"))  
      .alias("motif_qualite")  
)
```

- premier cas vrai = valeur retenue
- lecture proche d'un if / elif / else
- pratique pour expliquer une anomalie ligne par ligne
- point de vigilance : Polars évalue les expressions de branche indépendamment

Vérifier l'effet réel du nettoyage

```
df_clean = df.filter(pl.col("ligne_valide"))

avant = df.select(pl.len().alias("nb_lignes"))
apres = df_clean.select([
    pl.len().alias("nb_lignes"),
    pl.col("ligne_valide").sum().alias("nb_valides"),
])
```

- nombre de lignes
- nombre d'anomalies
- nombre de lignes valides
- évolution des statuts

Exercice J2-A - Construire un mini diagnostic qualité

Consigne :

- compter les lignes
- compter les anomalies
- compter les lignes valides
- produire une synthèse courte

Correction J2-A

```
diagnostic = df.groupby("statut_normalise").agg([
    pl.len().alias("nb_lignes"),
    pl.col("ligne_valide").sum().alias("nb_valides"),
    pl.col("montant_net").sum().alias("montant_total"),
]).sort("nb_lignes", descending=True)
```

- group_by
- agg
- indicateurs simples
- sortie compréhensible pour un métier

Pourquoi un graphique ici

- rendre un résultat lisible immédiatement
- aider à expliquer la qualité des données
- voir vite les déséquilibres

Passer d'un diagnostic à un graphique

```
df_chart = df.groupby("statut_normalise").agg(  
    pl.len().alias("nb_lignes")  
)
```

- un graphique part d'une table déjà agrégée
- `df.plot` n'est pas là pour nettoyer les données
- le nettoyage et les comptages se font d'abord avec Polars

Graphique 1 - Répartition des statuts

```
chart = (  
    df_chart.plot.bar(  
        x="statut_normalise",  
        y="nb_lignes",  
        color="statut_normalise",  
    )  
    .properties(width=500, title="Repartition des statuts")  
    .configure_scale(zero=False)  
    .configure_axisX(labelAngle=0)  
)  
chart.encoding.x.title = "Statut"  
chart.encoding.y.title = "Nombre de lignes"  
chart
```

- nombre de lignes par statut
- lecture rapide d'un volume par catégorie

Préparer la table des anomalies

```
df_anomalies = pl.DataFrame(  
    {  
        "type_anomalie": ["email_invalide", "montant_invalide", "date_invalide"],  
        "nb_lignes": [12, 5, 3],  
    }  
)
```

- un graphique a besoin d'une table simple
- ici : une catégorie + un volume

Graphique 2 - Nombre d'anomalies

```
chart = (  
    df_anomalies.plot.bar(  
        x="type_anomalie",  
        y="nb_lignes",  
        color="type_anomalie",  
    )  
    .properties(width=500, title="Nombre d anomalies")  
    .configure_scale(zero=False)  
    .configure_axisX(labelAngle=0)  
)  
chart.encoding.x.title = "Type d anomalie"  
chart.encoding.y.title = "Nombre de lignes"  
chart
```

- anomalies par type
- visualisation des problèmes prioritaires

Préparer la volumétrie par catégorie

```
df_categories = df.groupby("categorie").agg(  
    pl.col("montant_net").sum().alias("montant_total")  
)
```

- table d'entrée simple
- une catégorie, un total

Graphique 3 - Volumétrie par catégorie

```
chart = (  
    df_categories.plot.bar(  
        x="categorie",  
        y="montant_total",  
        color="categorie",  
    )  
    .properties(width=500, title="Volumetrie par categorie")  
    .configure_scale(zero=False)  
    .configure_axisX(labelAngle=0)  
)  
chart.encoding.x.title = "Categorie"  
chart.encoding.y.title = "Montant total"  
chart
```

- catégorie métier
- volume ou montant associé
- aide à prioriser les contrôles

Interpréter un graphique métier

- regarder les volumes
- regarder les écarts
- ne pas surinterpréter
- relier toujours au fichier source

Exercice J2-B - Produire 2 graphiques utiles

Consigne :

- un graphique de statuts
- un graphique d'anomalies ou de volumétrie
- expliquer en une phrase ce qu'ils montrent

Correction J2-B

```
chart = (  
    df_chart.plot.bar(  
        x="statut_normalise",  
        y="nb_lignes",  
        color="statut_normalise",  
        tooltip=["statut_normalise", "nb_lignes"],  
    )  
    .properties(width=500, title="Repartition des statuts")  
    .configure_scale(zero=False)  
    .configure_axisX(labelAngle=0)  
)  
chart.encoding.x.title = "Statut"  
chart.encoding.y.title = "Nombre de lignes"  
chart
```

- graphique lisible
- titre clair
- axes compréhensibles
- utilité métier explicite

Quand le fichier d'entrée est un Excel

```
df = pl.read_excel("input/entree.xlsx")
```

- utile quand le métier travaille d'abord sous Excel

Ce qu'il faut installer pour lire Excel

- nécessaire pour la lecture Excel dans ce cours
- à installer avant la session
- à tester sur les postes avant le jour J

Quand le livrable final doit être un Excel

```
df.write_excel("output/resultat.xlsx")
```

- sortie métier fréquente

Ce qu'il faut installer pour écrire Excel

- nécessaire pour l'écriture Excel
- à intégrer dans l'environnement standard du cours

Quand sortir en CSV, quand sortir en Excel

- CSV : traitement, échange simple, robustesse
- Excel : restitution métier, lecture humaine, diffusion

Exercice J2-C - Lire un Excel et réécrire une sortie propre

Consigne :

- lire un fichier Excel
- appliquer une transformation simple
- réécrire une sortie propre

Correction J2-C

```
import polars as pl

df = pl.read_excel("input/entree.xlsx")

result = df.with_columns(
    pl.col("email").str.strip_chars().str.to_lowercase().alias("email_clean")
)

result.write_excel("output/resultat.xlsx")
```

- lecture Excel
- transformation simple et lisible
- export rejouable vers Excel

Quand quitter le notebook

```
# notebook : exploration
df = pl.read_csv("input/clients.csv")
df.head()

# script : production
output_df = transform_clients(df)
output_df.write_csv("output/resultat.csv")
```

- quand la logique devient stable
- quand il faut rejouer
- quand il faut partager un livrable
- quand il faut industrialiser

Stabiliser le traitement dans un vrai script

```
def load_input(path: str) -> pl.DataFrame:  
    return pl.read_csv(path)
```

```
def transform_clients(df: pl.DataFrame) -> pl.DataFrame:  
    return df.with_columns(...)
```

- une fonction = une responsabilité
- améliorer la lisibilité
- faciliter le test manuel

Script minimal rejouable de bout en bout

```
def load_input(path: str) -> pl.DataFrame:
    return pl.read_csv(path)

def transform_clients(df: pl.DataFrame) -> pl.DataFrame:
    return df.with_columns(...)

def main() -> None:
    df = load_input("input/clients.csv")
    result = transform_clients(df)
    result.write_csv("output/resultat.csv")

if __name__ == "__main__":
    main()
```

- centraliser l'enchaînement
- séparer chargement, transformation et export
- rendre le script exécutable directement

Arborescence input / output / secret / libs

```
projet/  
  input/  
  output/  
  secret/  
  libs/  
  main.py
```

- input/ : données d'exemple
- output/ : résultats
- secret/ : hors contexte
- libs/ : utilitaires génériques

Conserver libs générique et métier dans main.py

- `libs/` pour ce qui se réutilise
- `main.py` pour la logique locale du script
- ne pas tout mettre dans `libs/`

Exemple : transformer une cellule notebook en fonction

```
# notebook
df = df.with_columns(
    pl.col("email").str.to_lowercase().alias("email_clean")
)

# script
def add_clean_email(df: pl.DataFrame) -> pl.DataFrame:
    return df.with_columns(
        pl.col("email").str.to_lowercase().alias("email_clean")
    )
```

- prendre un bout de code stable
- l'extraire dans une fonction nommée
- rappeler l'intérêt : rejouer, lire, corriger

Exercice J2-D - Transformer une logique notebook en script

Consigne :

- prendre une transformation validée en notebook
- la mettre dans une fonction
- l'appeler depuis `main()`

Correction J2-D

```
def transform_clients(df: pl.DataFrame) -> pl.DataFrame:
    return df.with_columns(
        pl.col("email").str.strip_chars().str.to_lowercase().alias("email_clean")
    )

def main() -> None:
    df = pl.read_csv("input/clients.csv")
    transform_clients(df).write_csv("output/resultat.csv")

if __name__ == "__main__":
    main()
```

- fonction claire
- point d'entrée propre
- lecture du fichier
- export du résultat

Quand le résultat n'est pas celui attendu

- un script ne fait pas toujours ce qu'on croit
- lire le code ne suffit pas toujours
- le debugger évite de travailler à l'aveugle

Arrêter l'exécution au bon moment

```
def normalize_email(email: str) -> str:  
    return email.strip().lower()
```

```
email = normalize_email(" CONTACT@X.FR ")  
print(email)
```

- arrêter l'exécution à un point précis
- regarder l'état intermédiaire

Avancer sans perdre le fil

- avancer ligne par ligne
- entrer dans une fonction si nécessaire
- comprendre où la logique dévie

Vérifier la donnée au moment du bug

```
current_email = row["email"]  
email_clean = current_email.strip().lower()
```

- lire la valeur courante
- vérifier si le problème vient de l'entrée ou de la logique

Lire une erreur avant de corriger

FileNotFoundError: [Errno 2] No such file or directory:
'input/clients.csv'

- identifier le message
- repérer la ligne
- comprendre si le problème vient du code
- comprendre si le problème vient de la donnée
- comprendre si le problème vient du chemin de fichier

Exercice J2-E - Déboguer un script qui sort un mauvais résultat

Consigne :

- poser un breakpoint
- inspecter une variable
- identifier la ligne fautive
- expliquer la correction

Correction J2-E

```
# bug observe au debugger  
def normalize_email(email: str) -> str:  
    return email.lower()
```

```
# correction  
def normalize_email(email: str) -> str:  
    return email.strip().lower()
```

- breakpoint posé avant la normalisation
- valeur brute inspectée : espaces parasites visibles
- cause du bug identifiée
- correction locale et explicite

Atelier Jour 2 - Sortir un fichier propre + un fichier anomalies

Objectif :

- produire un résultat métier propre
- séparer les anomalies
- disposer d'un script exécutable

Squelette atelier J2

- lecture du fichier
- transformation
- diagnostic qualité
- export résultat
- export anomalies

Livrable Jour 2

- notebook de contrôle
- script structuré
- sortie propre
- fichier d'anomalies

Quiz flash Jour 2

- rôle de `group_by`
- rôle de `value_counts()`
- rôle de `df.plot`
- rôle de `when / then / otherwise`
- différence notebook / script
- rôle du debugger

Récapitulatif Jour 2

- contrôler
- compter rapidement des catégories avec `value_counts()`
- visualiser
- créer des colonnes conditionnelles métier
- lire / écrire Excel
- structurer un script
- déboguer

Jour 3

**Volume + agrégations + jointure
FX + API réelle + bonus debugger**

Jour 3 - objectifs

- lire un dataset de 10 000+ lignes sans se perdre
- faire plus d'agrégations utiles
- joindre des taux de change sur date + country
- enrichir avec une API réelle
- terminer par un bonus debugger séparé

Support du jour

Fichiers utilisés :

- `cours/jeux_de_donnees/ecommerce_pedagogique/input/commandes_volume_j3.csv`
- `cours/jeux_de_donnees/ecommerce_pedagogique/input/daily_j3.csv`
- `cours/jeux_de_donnees/ecommerce_pedagogique/api/rest_countries_fallback_j3.json`
- `code/intermediaire/j3_volume_fx_api.py`
- `code/intermediaire/j3_bonus_debugger_bug.py`
- `code/intermediaire/j3_bonus_debugger_fix.py`

Ce qui change par rapport à J1 et J2

- le dataset est beaucoup plus grand : 10 500 lignes
- on ne lit plus tout à l'oeil
- on prépare vite des colonnes de travail
- on agrège avant et après enrichissement
- on garde le logging dans le script final, pas dans un chapitre à part

Pourquoi le volume change la lecture du DataFrame

Sur 30 lignes, on peut presque tout regarder.

Sur 10 500 lignes, on travaille autrement :

- volumétrie
- schéma
- nulls
- statistiques rapides
- agrégations ciblées

Lire un jeu de données plus grand

```
import polars as pl

df = pl.read_csv(
    "cours/jeux_de_donnees/ecommerce_pedagogique/input/commandes_volume_j3.csv"
)

print(df.shape)
print(df.head())
```

- premier réflexe : taille + aperçu
- pas de nettoyage lourd à ce stade

Premiers contrôles rapides sur gros volume

```
print(df.schema)
print(df.null_count())
print(df.describe())
```

```
preview = df.select([
    "commande_id",
    "date_commande",
    "categorie",
    "montant",
    "statut",
    "country",
]).head(10)
```

- `schema` : types inférés
- `null_count()` : trous visibles vite
- `describe()` : ordre de grandeur

Préparer des colonnes de travail

```
def parse_amount(raw: str | None) -> float | None:
    value = (raw or "").strip()
    if not value or value.upper() == "N/A":
        return None

    value = value.replace("€", "").replace("$", "").replace(" ", "")

    if "," in value and "." in value:
        if value.rfind(",") > value.rfind("."):
            value = value.replace(".", "").replace(",", ".")
        else:
            value = value.replace(",", ".")
    elif "," in value:
        value = value.replace(",", ".")

    try:
        return float(value)
    except ValueError:
        return None
```

- on prépare une colonne dérivée
- on ne détruit pas la colonne source montant

Parser les dates sans casser la source

```
def parse_date_expr(column_name: str) -> pl.Expr:
    return pl.coalesce(
        [
            pl.col(column_name).str.strptime(pl.Date, format="%Y-%m-%d", strict=False),
            pl.col(column_name).str.strptime(pl.Date, format="%d/%m/%Y", strict=False),
            pl.col(column_name).str.strptime(pl.Date, format="%Y/%m/%d", strict=False),
        ]
    )
```

- plusieurs formats possibles
- une seule colonne dérivée : `order_date`

Construire le DataFrame exploitable

```
df = (  
    df.with_columns(  
        [  
            pl.col("quantite").cast(pl.Int64, strict=False),  
            parse_date_expr("date_commande").alias("order_date"),  
            pl.col("montant")  
                .map_elements(parse_amount, return_dtype=pl.Float64)  
                .alias("montant_net"),  
            pl.col("statut")  
                .str.strip_chars()  
                .str.to_uppercase()  
                .alias("statut_normalise"),  
        ]  
    )  
    .with_columns(  
        pl.col("order_date").dt.strftime("%Y-%m").alias("mois_commande")  
    )  
)
```

- `map_elements()` reste utile quand la logique Python est plus simple
- à partir d'ici, on agrège sur les colonnes dérivées

Compter vite avec value_counts()

```
status_counts = (  
    df.select(pl.col("statut_normalise").value_counts(sort=True))  
    .unnest("statut_normalise")  
)
```

```
print(status_counts)
```

- parfait pour un premier contrôle métier
- ici : nombre de commandes par statut

CA par catégorie

```
ca_par_categorie = (  
    df.groupby("categorie")  
    .agg(  
        [  
            pl.len().alias("nb_commandes"),  
            pl.col("customer_id").n_unique().alias("nb_clients_uniques"),  
            pl.col("montant_net").sum().round(2).alias("montant_total"),  
            pl.col("montant_net").mean().round(2).alias("montant_moyen"),  
        ]  
    )  
    .sort("montant_total", descending=True)  
)
```

- sum
- mean
- n_unique
- sort

CA par pays

```
ca_par_pays = (  
    df.groupby("country")  
    .agg(  
        [  
            pl.len().alias("nb_commandes"),  
            pl.col("customer_id").n_unique().alias("nb_clients_uniques"),  
            pl.col("montant_net").sum().round(2).alias("montant_total"),  
        ]  
    )  
    .sort("montant_total", descending=True)  
)
```

- lecture business immédiate
- utile avant la jointure FX

CA par pays et catégorie

```
ca_par_pays_categorie = (  
    df.groupby(["country", "categorie"])  
    .agg(  
        [  
            pl.len().alias("nb_commandes"),  
            pl.col("customer_id").n_unique().alias("nb_clients_uniques"),  
            pl.col("montant_net").sum().round(2).alias("montant_total"),  
        ]  
    )  
    .sort(["country", "montant_total"], descending=[False, True])  
)
```

- agrégation plus riche
- meilleure préparation avant enrichissement externe

Exercice J3-A - Explorer et agréger le gros dataset

Consigne :

- lire `commandes_volume_j3.csv`
- créer `order_date`, `montant_net`, `statut_normalise`, `mois_commande`
- produire :
 - un `value_counts()` des statuts
 - un CA par categorie
 - un CA par country

Correction J3-A

```
import polars as pl

df = pl.read_csv(
    "cours/jeux_de_donnees/ecommerce_pedagogique/input/commandes_volume_j3.csv"
)

df = (
    df.with_columns(
        [
            parse_date_expr("date_commande").alias("order_date"),
            pl.col("montant")
                .map_elements(parse_amount, return_dtype=pl.Float64)
                .alias("montant_net"),
            pl.col("statut").str.strip_chars().str.to_uppercase().alias("statut_normalise"),
            pl.col("quantite").cast(pl.Int64, strict=False),
        ]
    )
    .with_columns(pl.col("order_date").dt.strftime("%Y-%m").alias("mois_commande"))
)

status_counts = (
    df.select(pl.col("statut_normalise").value_counts(sort=True))
    .unnest("statut_normalise")
)

ca_par_categorie = (
    df.group_by("categorie")
    .agg(pl.col("montant_net").sum().round(2).alias("montant_total"))
    .sort("montant_total", descending=True)
)

ca_par_pays = (
    df.group_by("country")
    .agg(pl.col("montant_net").sum().round(2).alias("montant_total"))
    .sort("montant_total", descending=True)
)
```

Pourquoi préparer la jointure devise

Le besoin métier est simple :

- le montant source est traité comme une base USD
- on veut un montant converti selon le pays et la date
- la clé de jointure n'est pas seulement le pays
- il faut date + country

Lire le sous-ensemble de daily.csv

```
rates_df = pl.read_csv(  
    "cours/jeux_de_donnees/ecommerce_pedagogique/input/daily_j3.csv"  
)
```

```
print(rates_df.head())  
print(rates_df.schema)
```

- daily_j3.csv garde le même schéma que la source daily.csv
- on a seulement réduit le volume pour le cours

Aligner les clés de jointure

```
rates_df = (  
    rates_df.with_columns(  
        [  
            parse_date_expr("Date").alias("order_date"),  
            pl.col("Country").alias("country"),  
            pl.col("Exchange rate").cast(pl.Float64).alias("exchange_rate"),  
        ]  
    )  
    .select(["order_date", "country", "exchange_rate"])  
    .drop_nulls()  
)
```

- même nom de colonnes des deux côtés
- mêmes types des deux côtés

Joindre daily.csv sur date et pays

```
fx_df = df.join(  
    rates_df,  
    on=["order_date", "country"],  
    how="left",  
)
```

- jointure gauche : on garde toutes les commandes
- si le taux manque, on le voit tout de suite

Calculer les montants convertis

```
fx_df = fx_df.with_columns(  
    [  
        (pl.col("montant_net") * pl.col("exchange_rate"))  
        .round(2)  
        .alias("montant_converti"),  
        pl.col("exchange_rate").is_null().alias("fx_manquant"),  
    ]  
)
```

- nouvelle colonne métier
- plus une colonne de contrôle

Agrégations utiles après la jointure FX

```
fx_resume = (  
    fx_df.groupby("country")  
    .agg(  
        [  
            pl.len().alias("nb_commandes"),  
            pl.col("fx_manquant").sum().alias("nb_fx_manquants"),  
            pl.col("montant_converti").sum().round(2).alias("montant_converti_total"),  
        ]  
    )  
    .sort("montant_converti_total", descending=True)  
)
```

- on vérifie la qualité de la jointure
- on lit déjà une première synthèse convertie

Exercice J3-B - Jointure devise

Consigne :

- lire `daily_j3.csv`
- préparer `order_date, country, exchange_rate`
- joindre sur `order_date + country`
- créer `montant_converti`
- compter les lignes avec taux manquant

Correction J3-B

```
rates_df = (  
    pl.read_csv("cours/jeux_de_donnees/ecommerce_pedagogique/input/daily_j3.csv")  
    .with_columns(  
        [  
            parse_date_expr("Date").alias("order_date"),  
            pl.col("Country").alias("country"),  
            pl.col("Exchange rate").cast(pl.Float64).alias("exchange_rate"),  
        ]  
    )  
    .select(["order_date", "country", "exchange_rate"])  
)  
  
fx_df = (  
    df.join(rates_df, on=["order_date", "country"], how="left")  
    .with_columns(  
        [  
            (pl.col("montant_net") * pl.col("exchange_rate")).round(2).alias("montant_converti"),  
            pl.col("exchange_rate").is_null().alias("fx_manquant"),  
        ]  
    )  
)  
  
nb_fx_manquants = fx_df.filter(pl.col("fx_manquant")).height  
print(nb_fx_manquants)
```

Pourquoi enrichir avec une API réelle

- le CSV local ne contient pas tout
- on veut `region` et `subregion`
- l'appel est simple, en lecture seule
- on garde un fallback local si la salle n'a plus de réseau

Appeler Rest Countries avec requests

```
import requests

endpoint = "https://restcountries.com/v3.1/all?fields=name,region,subregion"
response = requests.get(endpoint, timeout=20)
response.raise_for_status()
payload = response.json()
```

- un seul appel
- timeout obligatoire
- `raise_for_status()` obligatoire

Exemple minimal du JSON API

```
[
  {
    "name": {"common": "Australia"},
    "region": "Oceania",
    "subregion": "Australia and New Zealand",
  },
  {
    "name": {"common": "Canada"},
    "region": "Americas",
    "subregion": "North America",
  },
]
```

- on ne garde que les champs utiles
- pas besoin de tout charger mentalement

Transformer le JSON API en DataFrame

```
def countries_payload_to_frame(payload: list[dict]) -> pl.DataFrame:
    rows = []
    for item in payload:
        common_name = item.get("name", {}).get("common")
        if not common_name:
            continue
        rows.append(
            {
                "country": common_name,
                "region": item.get("region") or "Unknown",
                "subregion": item.get("subregion") or "Unknown",
            }
        )

    return pl.DataFrame(rows).unique(subset=["country"], maintain_order=True)
```

- JSON -> liste de dictionnaires
- liste de dictionnaires -> DataFrame

Gérer le fallback local proprement

```
import json
from pathlib import Path
```

```
def load_fallback_countries(path: str) -> list[dict]:
    with Path(path).open(encoding="utf-8") as handle:
        return json.load(handle)
```

- si l'API tombe, on ne bloque pas la salle
- le fallback ne remplace pas la logique API, il sécurise la démo
- il contient aussi la ligne pédagogique Euro absente de l'API réelle

Joindre l'enrichissement API

```
countries_df = countries_payload_to_frame(payload)

enriched_df = (
    fx_df.join(countries_df, on="country", how="left")
    .with_columns(
        [
            pl.col("region").fill_null("Unknown"),
            pl.col("subregion").fill_null("Unknown"),
        ]
    )
)
```

- clé de jointure simple : country
- on garde un Unknown si une valeur n'existe pas

Total converti par région après enrichissement API

```
ca_par_region = (  
    enriched_df.groupby("region")  
    .agg(  
        pl.col("montant_converti")  
        .sum()  
        .round(2)  
        .alias("montant_converti_total")  
    )  
    .sort("montant_converti_total", descending=True)  
)
```

- c'est l'agrégation que les étudiants ont explicitement demandée

Agrégation finale du mini-projet

```
final_df = (  
    enriched_df.groupby(["mois_commande", "region", "country", "categorie"])  
    .agg(  
        [  
            pl.len().alias("nb_commandes"),  
            pl.col("customer_id").n_unique().alias("nb_clients_uniques"),  
            pl.col("montant_net").sum().round(2).alias("montant_total"),  
            pl.col("montant_converti")  
                .sum()  
                .round(2)  
                .alias("montant_converti_total"),  
        ]  
    )  
    .sort(["mois_commande", "region", "country", "categorie"])  
)
```

- c'est le livrable analytique final de J3

Exercice J3-C - Enrichir avec l'API et agréger

Consigne :

- appeler Rest Countries
- transformer le JSON en table Polars
- joindre region et subregion
- calculer le total converti par region
- produire l'agrégation finale par mois_commande, region, country, categorie

Correction J3-C

```
endpoint = "https://restcountries.com/v3.1/all?fields=name,region,subregion"
response = requests.get(endpoint, timeout=20)
response.raise_for_status()
payload = response.json()

countries_df = countries_payload_to_frame(payload)

enriched_df = (
    fx_df.join(countries_df, on="country", how="left")
    .with_columns(
        [
            pl.col("region").fill_null("Unknown"),
            pl.col("subregion").fill_null("Unknown"),
        ]
    )
)

ca_par_region = (
    enriched_df.group_by("region")
    .agg(pl.col("montant_converti").sum().round(2).alias("montant_converti_total"))
    .sort("montant_converti_total", descending=True)
)

final_df = (
    enriched_df.group_by(["mois_commande", "region", "country", "categorie"])
    .agg(
        [
            pl.len().alias("nb_commandes"),
            pl.col("customer_id").n_unique().alias("nb_clients_uniques"),
            pl.col("montant_net").sum().round(2).alias("montant_total"),
            pl.col("montant_converti").sum().round(2).alias("montant_converti_total"),
        ]
    )
    .sort(["mois_commande", "region", "country", "categorie"])
)
```

Script final compressé : interface CLI

```
parser = argparse.ArgumentParser(description="J3 volume + FX + API + aggregations")
parser.add_argument("--orders", default="cours/jeux_de_donnees/ecommerce_pedagogique/input/commandes_volume_j3.csv")
parser.add_argument("--rates", default="cours/jeux_de_donnees/ecommerce_pedagogique/input/daily_j3.csv")
parser.add_argument("--output", default="cours/jeux_de_donnees/ecommerce_pedagogique/output/j3_aggregate_final.csv")
parser.add_argument("--countries-api", default="https://restcountries.com/v3.1/all?fields=name,region,subregion")
parser.add_argument("--api-fallback", default="cours/jeux_de_donnees/ecommerce_pedagogique/api/rest_countries_fallback_j3.json")
args = parser.parse_args()
```

- interface simple
- tout passe par des fichiers .py

Script final compressé : logging minimal

```
import logging

logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
logging.info("Chargement du gros dataset commandes")
logging.info("Lignes lues: %s", orders_df.height)
logging.info("Jointure devise sur date + pays")
logging.info("Lignes avec taux manquant: %s", fx_df.filter(pl.col("fx_manquant")).height)
```

- démarrage
- volumétrie
- erreur API si besoin
- pas plus

Script final compressé : enchaînement complet

```
orders_df = load_orders(args.orders)
rates_df = load_rates(args.rates)
fx_df = join_exchange_rates(orders_df, rates_df)

countries_df = fetch_countries_dataframe(
    args.countries_api,
    args.api_fallback,
    set(fx_df.get_column("country").unique().to_list()),
)

enriched_df = join_countries(fx_df, countries_df)
final_df = final_aggregate(enriched_df)

final_df.write_csv(args.output)
```

- script rejouable
- pipeline lisible
- support principal : `code/intermediaire/j3_volume_fx_api.py`

Atelier final J3

Objectif :

- partir de `commandes_volume_j3.csv`
- joindre les taux de change
- appeler l'API réelle
- produire une agrégation finale exploitable
- exporter depuis un script `.py`

Livrables J3

- `code/intermediaire/j3_volume_fx_api.py`
- `code/intermediaire/j3_bonus_debugger_bug.py`
- `code/intermediaire/j3_bonus_debugger_fix.py`
- `cours/jeux_de_donnees/ecommerce_pedagogique/input/ commandes_volume_j3.csv`
- `cours/jeux_de_donnees/ecommerce_pedagogique/input/daily_j3.csv`
- un export final agrégé
- un export détaillé enrichi

Bonus - debugger

- bloc séparé de 20 à 30 minutes
- pas intégré au mini-projet principal
- support : un script Python pur volontairement faux
- objectif : comprendre pourquoi la jointure produit un mauvais résultat

Pourquoi un bonus sans Polars

- le but n'est pas de déboguer une expression Polars
- le but est de suivre une logique Python simple
- on veut voir :
 - une liste de dictionnaires
 - une clé de jointure
 - un dictionnaire de lookup
 - une agrégation Python lisible
- c'est plus adapté à un bonus debugger court

Le script volontairement faux

```
def parse_order_date_bad(raw: str | None) -> str | None:
    value = (raw or "").strip()
    if not value:
        return None

    try:
        return datetime.strptime(value, "%d/%m/%Y").date().isoformat()
    except ValueError:
        return None

def load_orders(path: Path) -> list[dict]:
    rows = []
    with path.open(newline="", encoding="utf-8") as handle:
        reader = csv.DictReader(handle)
        for row in reader:
            rows.append(
                {
                    "commande_id": row["commande_id"],
                    "country": row["country"],
                    "date_commande": row["date_commande"],
                    "order_date": parse_order_date_bad(row["date_commande"]),
                    "montant_net": parse_amount(row["montant"]),
                }
            )
    return rows
```

- le bug n'est pas dans join
- le bug est dans la préparation de la clé

Où mettre les breakpoints

- juste après `load_orders()`
- dans la boucle qui prépare `order_date`
- juste avant `rates_lookup.get(lookup_key)`
- juste avant l'addition dans l'agrégat final

Ce qu'on inspecte dans le debugger

- `row["date_commande"]`
- `order["order_date"]`
- `lookup_key`
- `rate`
- `summary[country]`

Correction du bug du bonus

```
def parse_order_date(raw: str | None) -> str | None:
    value = (raw or "").strip()
    if not value:
        return None

    for fmt in ("%Y-%m-%d", "%d/%m/%Y", "%Y/%m/%d"):
        try:
            return datetime.strptime(value, fmt).date().isoformat()
        except ValueError:
            pass

    return None
```

- on corrige la clé
- la jointure redevient cohérente
- l'agrégation finale retrouve un ordre de grandeur crédible

Script corrigé du bonus

- fichier : `code/intermediaire/j3_bonus_debugger_fix.py`
- même structure que le script faux
- une seule différence importante :
 - `parse_order_date()` gère `YYYY-MM-DD`, `DD/MM/YYYY`, `YYYY/MM/DD`
- intérêt pédagogique :
 - comparer deux scripts presque identiques
 - isoler exactement la cause du bug

Avant / après correction

Avant :

```
Australia,2308,2298,45922.80
Canada,2496,2482,43210.82
Euro,1257,1247,21246.18
Japan,2104,2101,550974.34
United Kingdom,2335,2325,12333.47
```

Après :

```
Australia,2308,23,6591974.39
Canada,2496,34,6349000.90
Euro,1257,15,1867413.98
Japan,2104,23,417678573.25
United Kingdom,2335,25,2535626.87
```

- le nombre de lignes non converties chute fortement
- les montants convertis changent d'ordre de grandeur
- le debugger doit amener les étudiants à expliquer pourquoi

Récapitulatif Jour 3

- gros dataset lu proprement
- agrégations locales plus riches
- jointure devise sur date + country
- appel API réel avec fallback local
- agrégation finale exploitable
- bonus debugger séparé

Évaluation finale

- sait lire un CSV et un Excel avec Polars
- sait créer des colonnes dérivées
- sait produire un graphique simple
- sait transformer un notebook en script
- sait appeler une API simple
- sait utiliser logging

Fin de formation